

ORION — Detailed Project Idea Doc

2026-09-18 · @Harshit Anand

Executive Summary

ORION (Autonomous Robustness Evaluation Platform; Python package `arep`) is a deterministic, statistically rigorous, CPU-only, cloud-native SaaS platform for evaluating the safety of autonomous-driving planning and control models. A team submits a model, picks a scenario suite, and gets back a statistically defensible safety report — composite score, four sub-scores, pass/fail verdict — through a browser or API, with no install and no GPU.

The core bet: CARLA and its peers are simulators built for photorealistic sensor data; ORION is a testing laboratory built for one job — telling a team, with confidence intervals, whether their model regressed. What ORION sells is certainty, not screenshots.

ORION is pre-revenue and mid-build. A production-readiness review on 2026-06-12 found the core simulation engine (deterministic PCG64 seeding, bicycle + Pacejka tire physics, SAT collision detection, Wilson/t-distribution statistics) solid, but surfaced 13 defects in the SaaS layer — 4 of them critical security holes (an RCE path in model upload, a forgeable JWT secret, wide-open CORS, tokens in localStorage) and one that fakes a scored metric. The project is now in Phase 0 of a six-phase, roughly 10.5-month roadmap for a two-person team, closing every one of those defects before any customer-facing or revenue feature ships. Secrets hardening (0.1) is done; model sandboxing (0.2) is next.

Vision & Honest Positioning

ORION is a cloud-native, subscription SaaS platform for AV safety evaluation. A team submits a model, chooses a scenario suite, and receives statistically rigorous safety reports — without installing anything, without owning a GPU cluster, and without hand-writing test cases.

Is:

- A deterministic, statistically rigorous, CI-friendly evaluation harness for planning and control stacks that consume ground-truth state.
- The cheapest path to "did my new model version regress on safety?", answered with confidence intervals, on every pull request.
- Reproducible by construction — same seed, same run, every time, auditable.

Is Not (and must not pretend to be):

- **Not a perception testbed.** No LiDAR, camera, or radar simulation; models that need sensor input can't be evaluated here yet. Target users are planning/control teams, RL researchers, motion-prediction teams, and education.
- **Not a certification tool.** No ISO 26262 ASIL mapping, no ISO 21448 (SOTIF) ODD coverage argument, no HARA traceability — an engineering regression tool, not a homologation tool, until the Phase 4 standards-alignment work lands.
- **Not photorealistic.** A Bloomberg Terminal, not an Unreal Engine — never competing with CARLA or DRIVE Sim on rendering.

Dimension	CARLA	ORION
Primary value	Photorealistic sensor data for training	Rigorous safety evaluation + scores
Infrastructure	GPU workstation, local install	Browser-based SaaS, CPU cloud
Evaluation rigor	Manual, no statistical framework	Automated, statistically rigorous
CI/CD integration	Not possible (Unreal Engine)	First-class feature
Target user	Perception ML engineers	Planning/control + safety engineers
Pricing model	Free/open-source, self-hosted	Subscription (run credits per tier)

Problem & Market Opportunity

No tool on the market today combines cloud-native SaaS delivery, CPU-only execution, and statistical evaluation rigor in one product. The two closest competitors, Applied Intuition and Foretellix, are enterprise-only with \$200K–\$2M+ contracts and no self-service path — inaccessible to mid-market AV teams. Open-source tools (CARLA, CommonRoad, Waymax) are technically capable but demand heavy DevOps investment, have no hosted service, and produce no managed statistical report.

Target customers:

- Software-first AV teams (robotaxi, ADAS, trucking, drone delivery) with a trained driving model that needs certifying before deployment.
- Safety validation engineers who need evidence of model robustness for regulatory submissions or internal safety boards.
- Platform teams at OEMs building internal evaluation infrastructure, who would otherwise spend ~18 months building custom tooling.

- Research groups moving from academic evaluation (CommonRoad, Waymax) to production-grade safety reporting.

Not the target customer:

- Perception engineers training camera or LiDAR models (CARLA, NVIDIA DRIVE Sim).
- RL researchers training driving policies from scratch (highway-env, MetaDrive, Waymax).
- Hardware-in-loop ECU testing (IPG CarMaker, dSPACE).

Product Overview

A user submits a model (SDK upload as a cloudpickle artifact, or a registered Docker container), picks a built-in or custom scenario — or a batch of them — and runs it as a single evaluation, an async batch of up to hundreds of seeded runs, or a live streamed run they can watch in a 3D viewer. The platform returns a composite safety score, four sub-scores, a pass/fail verdict against fixed thresholds, and (once Phase 2 ships) confidence intervals and failure clustering.

Two-part architecture:

- `arep_implementation/` — the Python backend: FastAPI + a deterministic simulation core + the evaluation pipeline.
- `orion-frontend/` — React + Three.js + Vite frontend: dashboard UI and live 3D visualization.

Naming note: the platform is ORION; the underlying Python package is `arep`. Imports are always `from arep.*` — the two names are not interchangeable in code.

ORION is explicitly a **testing harness**, not a game engine or 3D simulator: a model is fed through parameterized scenarios hundreds of times and the platform returns statistical safety scores, not a rendered world to explore.

Technical Architecture

The simulation core is a 4-layer execution architecture (L1-L4), built around three non-negotiable design principles:

Determinism first — fixed timestep only (`dt = 0.02s` , 50 Hz); never `time.time()` , `datetime.now()` , or Python's `random` module. All randomness flows through a seeded `RandomManager` (`seed = master_seed + run_index`). Dependency versions are pinned (`numpy==1.26.0` , `scipy==1.11.3`) because upgrading them silently breaks determinism.

Immutable state — `WorldState` and `VehicleState` are always `.copy()` -ed before mutation; every `SimulationEngine.step()` returns a new `WorldState` rather than mutating the input.

A fixed simulation step order, never reordered: validate action → apply physics to ego → update dynamic objects → update traffic lights → check collisions → check other termination conditions → increment time.

Physics runs in two selectable modes: `PhysicsMode.KINEMATIC` (bicycle model, fast — used for batch runs) and `PhysicsMode.DYNAMIC` (Pacejka tire model with surface friction, for testing tire/surface behavior). Four surface types are modeled by friction coefficient: dry asphalt ($\mu=1.0$), wet asphalt ($\mu=0.5$), ice ($\mu=0.2$), gravel ($\mu=0.6$). Collision detection uses oriented bounding boxes with the separating axis theorem (SAT).

Layer	Technology	Key constraint
Python core	Python ≥ 3.10 , numpy==1.26.0, scipy==1.11.3, pyyaml==6.0.1	Version-locked for determinism
API	FastAPI + uvicorn, SQLAlchemy 2.0, SQLite (dev) / PostgreSQL (prod)	Auth via JWT (python-jose)
Frontend	React 18, Vite 5, React Router 6, Three.js r160, @react-three/fiber 8, Recharts 2	No TypeScript yet
Testing	pytest with <code>--tb=short</code> , pytest-cov	Run from <code>arep_implementation/</code>
Linting	black + ruff + mypy	All must pass before commit

Models plug in through a single abstract interface, `ModelInterface` (`predict(observation) -> Action`, `reset()`), and are always invoked through a `ModelWrapper` for timing and error logging — never called directly. `Action` carries three normalized fields in `[-1.0, 1.0]`: steering (± 0.5 rad max), throttle (0–3.0 m/s²), brake (0–8.0 m/s²).

Scenario Taxonomy & Methodology

The foundational rule: **one scenario equals one behavioral requirement**. Weather, lighting, and surface friction are never separate scenarios — they're parameterization modifiers on an existing one. A pedestrian crossing in rain is the same behavioral challenge as one in clear weather; the taxonomy discards combinatorial enumeration entirely.

Six behavioral challenge categories (60–120 core scenarios target):

Category	Code	Challenge
Longitudinal Control	LON	Speed, following distance, braking — threat from directly ahead
Lateral Control	LAT	Lane position, lane changes, encroachment response
Intersection Negotiation	INT	Right-of-way analysis and yielding at junctions
Vulnerable Road User	VRU	Pedestrians, cyclists, child models, irregular dynamic obstacles
Emergency/Anomalies	EMG	Out-of-distribution events — wrong-way drivers, debris
Multi-Agent	MLT	Multiple conflicting agents interacting simultaneously

Every scenario must fill a strict schema or it's rejected from the taxonomy: **SCN-ID** , scenario name, behavioral category, one-sentence behavioral requirement, threat agent (with explicit compliance behavior), ego right-of-way status, threat direction (same/oncoming/crossing/rear/merge), trigger type and exact numeric trigger condition, occlusion state, a mathematically provable success criterion, exhaustive failure conditions, a parameterization profile (weather, speed, modifier bounds), a real-world crash anchor (tied to NHTSA Pre-Crash Typology or Euro NCAP AEB tests), and an engine trigger code for the NPC behavior tree.

Representative scenarios already specified:

- **LON-003** — Lead vehicle brakes at 0.9g from 80 km/h with the cause of the stop occluded; ego must decelerate $\geq 0.3g$ within 0.8s. Anchored to NHTSA Type 14.
- **VRU-008** — A child (1.0 m/s walking speed) emerges from between parked cars, fully occluded until 15–25 m out; ego must detect and drop below 5 km/h before the crossing line within 0.5s of first visibility. Anchored to Euro NCAP AEB VRU (occluded child).
- **EMG-004** — A geofenced black-ice patch causes sudden friction loss, invisible to cameras; ego must stay within 15° of yaw slip and inside its lane. Anchored to NHTSA Type 01.
- **INT-005**, **LAT-002**, and **MLT-007** cover an unprotected left turn, a highway cut-in, and a construction-zone stop with a tailgating follower — each with the same rigor: numeric trigger thresholds, explicit pass/fail margins, a named crash anchor.

An appendix checklist enforces the discipline further: unique behavioral requirement, fully specified threat agent, mathematically measurable pass/fail, precise numeric trigger, no weather/time-of-day as a taxonomy axis, physical plausibility, a designated engine trigger code, a cited crash anchor, rigorously bounded parameterization, and graceful handling of inconclusive (timeout) outcomes.

Evaluation Metrics & Scoring

Four metric modules, each a typed result, combine into one composite score — this scoring methodology is ORION's core IP and the basis of its sales pitch: certainty, not screenshots.

Module	Class	Weight	What it measures
<code>safety.py</code>	<code>SafetyMetrics</code>	50%	Collision penalty + minimum TTC + fraction of critical-TTC steps
<code>compliance.py</code>	<code>ComplianceMetrics</code>	20%	Speed-limit adherence, lane keeping
<code>stability.py</code>	<code>StabilityMetrics</code>	15%	Control smoothness
<code>reactivity.py</code>	<code>ReactivityMetrics</code>	15%	Response latency to threats

`CompositeEvaluator` combines all four into a single `composite_score`. A model **passes** a scenario when, across N seeded runs, `collision_rate < 0.01` and `intervention_rate < 0.05`. Time-to-collision thresholds: `TTC_SAFE = 10.0s` scores 1.0; `TTC_CRITICAL = 2.0s` flags a critical step. The statistical aggregator additionally computes Wilson and t-distribution confidence intervals around these scores internally — not yet surfaced to the API or dashboard, which today show point estimates only (planned for Phase 2.1).

Metric weights are treated as frozen: changing `COLLISION_WEIGHT` (0.50), `MIN_TTC_WEIGHT` (0.30), or `CRITICAL_TTC_WEIGHT` (0.20) requires updating the specification document and every existing baseline. Two known defects sit in this layer today: lane compliance is currently a hardcoded stub (`lane_frac = 1.0`) rather than a real computation, and TTC uses a constant-velocity approximation that is optimistic under braking — both are scheduled fixes in Phase 0.5, tracked below in Known Defects.

Platform Capabilities

Multi-tenancy — organisations and API keys are first-class; every JWT carries `org_id` + `role`, and all evaluation, batch, job, and results routes are scoped by organisation.

Model submission — two paths: an SDK upload of a cloudpickle artifact, or registering a Docker container. The `orion-sdk` package ships an `OrionClient` and a CLI (`orion models`, `orion runs`, `orion keys`).

Async batch execution — Celery + Redis queue; submitting a batch atomically deducts run credits (row-locked), fans out N tasks, and returns in under 300 ms with live progress exposed via a status endpoint. The last task to finish triggers aggregation and flips the batch to completed; failed tasks refund a credit.

Live streaming — a WebSocket (`/ws/simulation/{run_id}`) streams tick-by-tick frames to a React-Three-Fiber 3D viewer, so a user can watch a run happen in real time — road, ego vehicle, NPCs, and an instrument-panel HUD (sim time, speed, g-force, metric bars, verdict badge).

API surface — a REST API under `/api` for auth, models, scenarios, single/batch evaluation, jobs, results, and live runs, plus the run-streaming WebSocket. Built-in reference models (`ConstantAction`, `EmergencyBrake`, `SimpleLaneKeep`, `Random`) exist to sanity-check the harness itself.

Design system — "Mission Control" — a binding, dark-default HUD aesthetic built on Chakra Petch (display), Saira (UI), and JetBrains Mono (all numerals). It explicitly bans glassmorphism-as-primary-surface, purple/violet gradients, glow-pulse animations, and the generic AI-startup hero — positioned deliberately as "Bloomberg Terminal, not Unreal Engine." The redesign (Phases A-I) is complete: tokens, theming, icon set, app shell, and every page conform.

Competitive Landscape

An April 2026 internal survey rated 19 competitors plus ORION across seven dimensions — evaluation focus, statistical rigor, cloud/SaaS, GPU-free, CI/CD integration, setup ease, and cost accessibility — each scored 1-10 relative to ORION's target use case (certifying an AV model's safety before deployment). ORION's Phase 1 (pre-launch) target of 7.9/10 already beats every surveyed competitor; the Phase 3 (full-platform) target is 9.0/10.

Tool	Type	Primary purpose	Overall (/10)
CARLA	Open source	Photorealistic 3D simulation	2.9
SUMO	Open source	Traffic flow simulation	5.4
Apollo / Dreamview	Open source	Full AD software stack	4.3
Waymax	Open source	ML research evaluation (JAX)	6.0

Tool	Type	Primary purpose	Overall (/10)
CommonRoad	Open source	Academic trajectory evaluation	6.6
highway-env	Open source	RL training environment	6.3
MetaDrive	Open source	ML/RL training simulator	5.9
SMARTS	Open source	Multi-agent RL environment	5.1
Scenic	Open source	Probabilistic scenario language	5.9
LGSVL (discontinued)	Open source	Unity-based simulation	3.1
Applied Intuition	Commercial	Enterprise simulation platform	6.4
Foretellix	Commercial	Safety coverage testing (M-SDL)	6.0
NVIDIA DRIVE Sim	Commercial	GPU sensor simulation (Omniverse)	3.0
IPG CarMaker	Commercial	Vehicle dynamics testing (OEM)	4.7
dSPACE ASM	Commercial	HIL/SIL hardware-in-loop	4.3
MATLAB ADT	Commercial	Toolbox simulation & analysis	5.4
Cognata	Commercial	Cloud 3D simulation platform	5.0
Ansys AVxcelerate	Commercial	Safety-critical simulation	4.7
Siemens Prescan	Commercial	Sensor & environment simulation	4.0
ORION (Phase 1, pre-launch)	Commercial SaaS	Statistical safety evaluation	7.9
ORION (Phase 3, full platform)	Commercial SaaS	Statistical safety evaluation	9.0

CARLA — the name most likely to come up in a demo. GPU-bound (no CI runner or CPU cloud instance can run it), evaluation is entirely manual with no statistical framework, and typical setup takes 2–4 days. Positioning: "CARLA generates the data you train on. ORION

certifies the model you built from it." ORION is not competing to replace CARLA for perception training — it's replacing the custom evaluation scripts teams build alongside it.

Applied Intuition — the best-funded, most capable competitor (\$300M+ raised), with cloud simulation, scenario authoring, and some statistical reporting. Enterprise-only pricing (\$200K–\$2M+/year, no self-service) makes it inaccessible to ORION's mid-market target. Their GPU-rendered simulation means high per-run cost; ORION's CPU-only architecture makes marginal run cost negligible. The single biggest competitive risk: if Applied Intuition ships an SMB-priced tier, competitive pressure on ORION rises sharply.

Foretellix — ORION's closest philosophical competitor, focused on safety certification via M-SDL (Measurable Scenario Description Language) coverage metrics rather than simulation fidelity, backed by Volvo Cars. It's a testing-orchestration layer that requires an existing simulator (CARLA, VTD) underneath it — not a complete, hosted platform — and has the same enterprise-only sales cycle as Applied Intuition.

The Four Moats

A moat is a structural advantage a competitor can't replicate quickly. ORION has four.

1. **CPU-only architecture** — no rendering engine, no GPU dependency, pure deterministic physics. A 100-scenario suite costs roughly \$0.50 in cloud compute on ORION versus \$50+ in GPU-hours for a CARLA-based equivalent. Competitors built on Unreal Engine or Omniverse (CARLA, NVIDIA DRIVE Sim, Cognata, Applied Intuition) cannot replicate this without rebuilding their simulation stack from scratch.
2. **Statistical rigor as the product itself, not an add-on** — the four-metric composite score with confidence intervals, failure clustering, and regression detection is built for safety-board presentations and regulatory submissions. Most simulators produce simulation output; ORION produces safety certificates.
3. **CI/CD integration as the primary distribution channel** — the planned GitHub Action / GitLab CI integration is the viral growth loop: every PR that shows an ORION safety check teaches a new engineer the brand, the same mechanism that made Codecov, Snyk, and Dependabot indispensable. CARLA can't put a status check on a PR; Applied Intuition's price point means only one budget-holder per company even sees the product.
4. **Accessible pricing with a genuine free tier** — every commercial competitor (Applied Intuition, Foretellix, Ansys, IPG) runs an enterprise sales motion with no free tier and no self-service signup. A free tier (50 runs/month) at launch makes ORION the default answer to "how do I evaluate my model right now?" for every AV engineer who can't get a six-figure tool approved.

Current Status & Known Defects

Built and working (review-verified 2026-06-12): the simulation core — kinematic + Pacejka dynamic physics, SAT collision, deterministic PCG64 seeding, Wilson/t-distribution statistics; live streaming (`WS /ws/simulation/{run_id}` + the R3F viewer); multi-tenancy (orgs, roles, API keys, org-scoped routes); model submission (cloudpickle SDK path + Docker path); an async batch queue (Celery + Redis, atomic credit deduction); auth (bcrypt, hashed single-use reset tokens, superadmin role); an 18-scenario library spanning all six categories; CI (test + lint + Docker build), Alembic migrations, and clean git hygiene.

The same review found **13 defects**, four of them critical — this register is why Phase 0 exists and blocks all revenue work:

ID	Defect	Severity	Fixed in
D-01	Cloudpickle model upload is arbitrary code execution; the worker subprocess inherits the database URL	CRITICAL	0.2 (next)
D-02	JWT secret falls back to a hardcoded string; hardcoded DB creds; plaintext creds in docker-compose	CRITICAL	0.1 — done
D-03	CORS <code>allow_origins=["*"]</code> plus zero rate limiting on login/signup	CRITICAL	0.3
D-04	JWT stored in browser <code>localStorage</code> ; signup auto-activates with no email verification	CRITICAL	0.4
D-05	Lane compliance hardcoded to <code>1.0</code> — the lane-keeping score is fake	HIGH	0.5
D-06	<code>time.time()</code> inside the simulation tick — breaks the determinism guarantee and frame hashing	HIGH	0.5
D-07	<code>/models/</code> and <code>/scenarios/</code> routes unauthenticated — cross-org information disclosure	HIGH	0.3
D-08	Celery <code>max_retries=0</code> — a transient failure kills the run and the customer eats the cost	HIGH	0.6
D-09	No CI coverage gate; WebSocket, admin, billing, and partial-refund paths untested	HIGH	0.6

ID	Defect	Severity	Fixed in
D-10	No frontend error boundaries or 404 page; <code>OrgProvider</code> written but never mounted; 5 stub pages	MED	0.6
D-11	TTC uses a constant-velocity approximation — optimistic in critical braking scenarios	MED	0.5 (doc), Phase 2 (fix)
D-12	Weight transfer uses the previous step's acceleration (off-by-one)	LOW	0.5
D-13	SQLite in dev vs. Postgres in prod — <code>FOR UPDATE</code> row locking is a no-op on SQLite, hiding races	MED	0.6

Only D-02 is closed as of today; D-01 (model sandboxing) is the current priority.

Roadmap

Six phases, ordered by dependency, sized for a two-person team — roughly 10.5 months total from Phase 0's start. Execution rule: each phase finishes entirely before the next begins.

Phase	Name	Duration	Outcome
0	Security & Score Integrity	~1 month	Platform is safe to put a customer on
1	SaaS Foundation (remainder)	~1.5 months	Real paying customers can use it
2	Evaluation Depth	~2.5 months	Genuinely better than CARLA for evaluation
3	CI/CD Integration	~1.5 months	The killer feature that closes enterprise deals
4	Ecosystem Expansion	~2.5 months	Broad compatibility, real-world scenarios
5	Visualization, Frontend Debt & DX	~1.5 months	Professional, polished, demo-worthy

Phase 0 closes the defect register above: secrets hardening (done), model sandboxing (next — cloudpickle is an RCE path), API hardening (CORS whitelist, rate limiting, auth on open routes), auth-flow integrity (email verification, httpOnly cookies, short-lived WS

tickets), score integrity (real lane compliance, removing the determinism-breaking timestamp, per-run frame hashing), and reliability/test gates (Celery retries, a CI coverage gate on Postgres, cross-org denial tests).

Phase 1 ships Stripe billing and the road-topology engine — today only a flat two-lane straight road exists, which blocks roughly 35% of the scenario library (every `INT-*`, `EMG-002`, and `MLT-*` scenario needs intersections or merge lanes).

Phase 2 is where ORION becomes genuinely better than CARLA for evaluation: surfacing the statistical confidence intervals the aggregator already computes, failure clustering and root-cause analysis, the adversarial scenario search engine (the core differentiator — not yet started), model comparison and regression reports, and deterministic replay — promoted from a Phase 5 polish item because "re-run the exact failure from its seed" is the single best demo of the determinism claim.

Phase 3 builds the GitHub Actions / GitLab CI integration identified as the primary growth lever (see The Four Moats).

Phase 4 adds a ROS2 bridge (so real AV stacks can connect), an OpenDRIVE map parser, expansion of the scenario library from 18 to 60, OpenSCENARIO 2.0 import/export, and the first steps toward an ISO 26262/21448 standards-alignment story for enterprise buyers.

Phase 5 covers 3D visualization polish (GLTF vehicle models, a full environment), dashboard polish, documentation, and frontend type safety and test coverage.

Current position: Phase 0, secrets hardening (0.1) done, model sandboxing (0.2) in progress next.

Business Model & Go-to-Market

Subscription SaaS priced on run credits per tier, billed through Stripe (scaffolded, not yet live — gated behind Phase 0). A genuine free tier (50 runs/month) is the acquisition wedge: it makes ORION the default answer to "how do I evaluate my model right now?" for engineers who can't get a six-figure enterprise tool approved, in a market where every commercial competitor runs an enterprise-only sales motion.

Strategic recommendations from the competitor analysis:

1. **Launch the free tier fast, right at Phase 1 completion.** Every week without it is a week an engineer discovers CommonRoad or starts building their own evaluation scripts instead.
2. **Win the GitHub Action race.** No competitor has a published, working GitHub Action for AV safety evaluation. Being the first search result for "autonomous driving CI/CD" makes organic growth self-sustaining.

3. **Partner with CARLA rather than compete against it.** Most target customers already use CARLA for training data. Position ORION as the evaluation layer that comes after it, and publish a CARLA-to-ORION integration guide with a worked conversion example.
4. **Publish the scoring methodology before launch.** Applied Intuition and Foretellix win enterprise trust through years of customer relationships; ORION can short-circuit that by publishing exactly how scores are calculated and why adversarial search finds edge cases, so engineers can verify the math themselves.
5. **Watch Applied Intuition's pricing page.** The only scenario where ORION loses the mid-market is Applied Intuition deciding to compete there. Their job postings for "growth" or "PLG" roles are the early warning sign.

Risks & Open Questions

Technical risk — the road topology engine (only a flat two-lane road exists today) blocks roughly 35% of the planned scenario library, including every intersection and multi-agent scenario, and isn't scheduled until Phase 1. The adversarial search engine — the core Phase 2 differentiator versus CARLA and the thing that would prove ORION finds edge cases competitors can't — hasn't been started. Sensor simulation is deliberately out of scope, which is a positioning strength but permanently caps the addressable market to planning/control teams and excludes any perception-model customer.

Market risk — Applied Intuition is the single biggest competitive threat: they have the funding and technical breadth to launch a mid-market tier at any time, and the honest read is that ORION only wins that segment if it establishes customer relationships and brand recognition first.

Execution risk — a ~10.5-month roadmap for a two-person team is aggressive, and the Phase 0 security work (four critical defects, one of them an active RCE path) must close completely before any revenue-generating feature ships — there's no partial-credit path to launch.

Immediate Next Steps

1. Close out Phase 0 in order: model sandboxing (D-01, the active RCE path) is next, then API hardening (D-03, D-07), then auth-flow integrity (D-04), then score integrity (D-05, D-06, D-11, D-12), then reliability and test gates (D-08, D-09, D-10, D-13).
2. Ship Stripe billing and the road-topology engine in Phase 1 — the latter unblocks roughly 35% of the scenario library that's currently written but can't run.

3. Prioritize deterministic replay early in Phase 2: it's the cheapest, highest-credibility demo of the core determinism claim, and it was previously buried under lower-value polish work.
4. Make the GitHub Action the first Phase 3 deliverable — it's the identified primary growth lever, and no competitor has shipped one yet.